Department of Computer Science

Faculty of Engineering, Built Environment & IT

University of Pretoria

COS 301: Software Engineering

Software Requirements and Design Specification

Capstone Project 2026 - Demo 1

Team Name: OmniTech

May 22, 2026

Contents

1 Introduction

Driving Tracker is an intelligent Android-based driving companion developed by OmniTech. The system aims to improve road safety, fuel efficiency, and vehicle maintenance by combining mobile sensors, GPS tracking, and vehicle diagnostics into a single integrated platform. Unlike existing solutions that focus only on navigation or diagnostics, Driving Tracker provides drivers with real-time feedback on driving behavior and vehicle health.

This document explores the software requirements and design specifications of the Driving Tracker application, as well as the architectural and technology requirements.

2 Project Owner

The project owner and client of the Driving Tracker is Gendac, and our industry mentors are Kevin Cowdrey, Sthabiso Jaffar, and Brandon Craytor.

3 Use Cases: Demo 1

3.1 Use Cases

1. A user starts and ends a trip, as well as view their trip summary.
2. A user adds a trusted contact.
3. A user is able to view weekly leaderboards, challenges, and as well as the badges they have.

3.2 Use Case Diagram

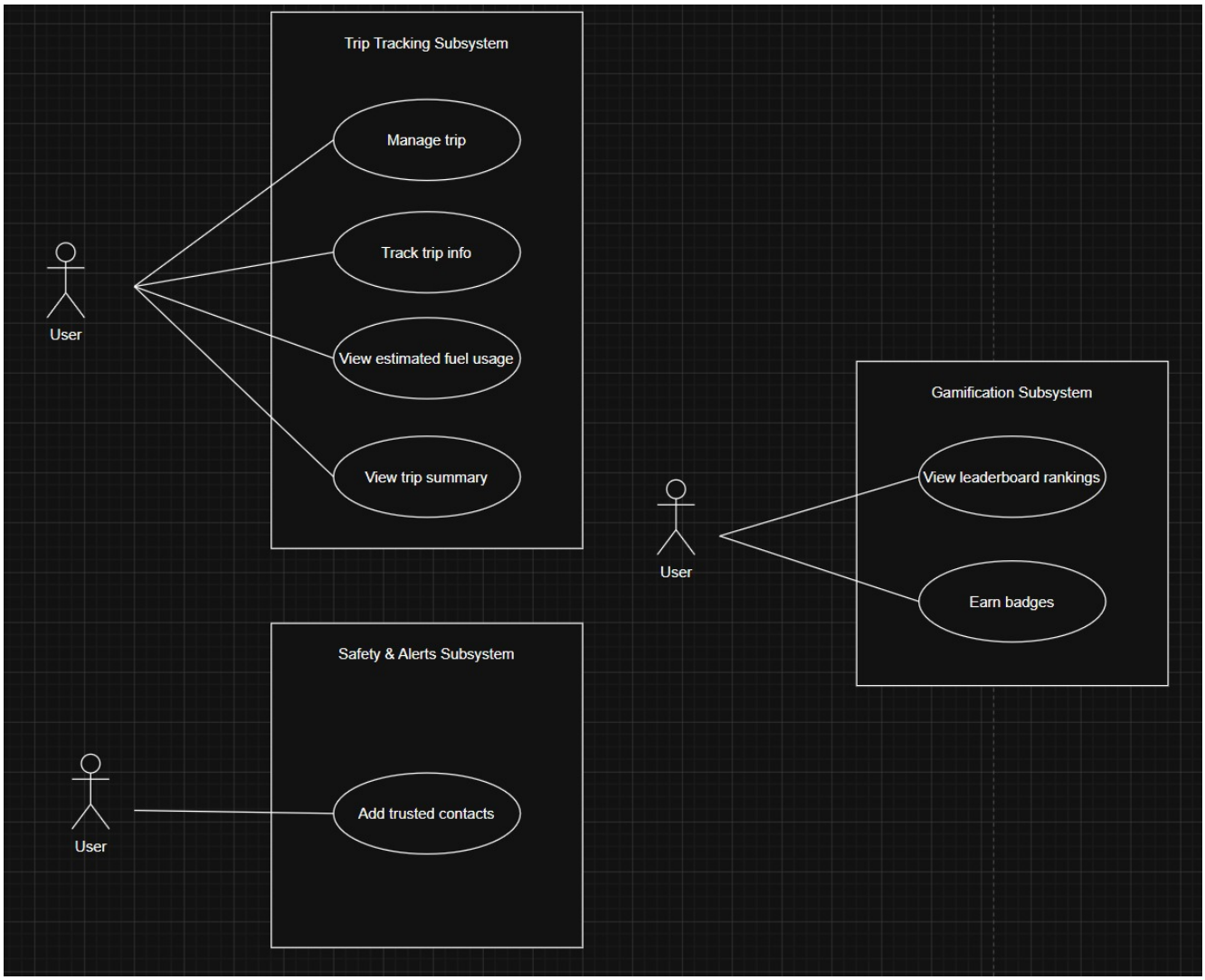

Figure 1: Demo 1 use cases

4 User Stories

4.1 User Management:

1. As a new user, I want to create an account using my email and password so that I can securely access the Driving Tracker platform.
2. As a registered user, I want to securely log into the application so that I can access my driving data and features.
3. As a user, I want to reset my password if I forget it so that I can regain access to my account.
4. As a user, I want to update my profile information so that my account details remain accurate.
5. As a user, I want to add trusted emergency contacts so that they can be notified during dangerous situations.

4.2 GPS Tracking and Trips:

1. As a driver, I want my trips and routes to be tracked in real time so that I can review my journeys later.
2. As a driver, I want to view my trip routes on an interactive map so that I can visualize where I travelled.
3. As a driver, I want to access my previous trips so that I can review my driving patterns over time.
4. As a driver, I want to replay completed trips on a map so that I can analyze my driving behavior.

4.3 Driving Behavior Monitoring:

1. As a driver, I want the system to detect harsh braking events so that I can improve my driving safety.
2. As a driver, I want the system to monitor my speed during trips so that I can avoid unsafe driving behavior.
3. As a driver, I want dangerous cornering events to be detected so that I can drive more safely.
4. As a driver, I want the system to detect possible crash events so that emergency actions can be triggered quickly. As a driver, I want to receive fatigue and rest-stop alerts during long trips so that I can avoid unsafe driving conditions.

4.4 OBD and Vehicle Diagnostics:

1. As a driver, I want to connect my vehicle to the app using a Bluetooth OBD 2 adapter so that vehicle diagnostics can be monitored.
2. As a driver, I want to monitor my vehicle's health in real time so that I can identify issues early.
3. As a driver, I want to view vehicle error codes so that I can understand potential mechanical problems.
4. As a driver, I want to receive maintenance warnings so that I can prevent serious vehicle failures.

4.5 Fuel Efficiency and Eco-Driving:

1. As a driver, I want the system to estimate fuel usage per trip so that I can better understand my fuel consumption.
2. As a driver, I want personalized eco-driving recommendations so that I can improve fuel efficiency.
3. As a driver, I want to view fuel efficiency trends over time so that I can monitor improvements in my driving habits.

4.6 Safety Scores and Analytics:

1. As a driver, I want to receive a safety score after each trip so that I can evaluate my driving performance.
2. As a driver, I want detailed driving analytics so that I can identify unsafe driving patterns.
3. As a driver, I want the system to classify my driving style so that I can better understand my driving behavior.

4.7 Gamification and Social Features:

1. As a driver, I want to earn badges and achievements so that safe and efficient driving feels rewarding.
2. As a driver, I want to participate in weekly driving challenges so that I stay motivated to improve.
3. As a driver, I want to compare my safety and efficiency scores with other users so that I can track my performance competitively.

4. As a driver, I want to compare my driving performance with users who have similar vehicle models so that the comparisons are more meaningful.

4.8 Emergency and Social Features:

1. As a driver, I want emergency alerts to be sent to trusted contacts during dangerous situations so that help can arrive quickly.
2. As a driver, I want to share my live trip status with my trusted contacts so that they can monitor my safety.

4.9 Fleet Management:

1. As a fleet manager, I want to monitor multiple drivers and vehicles so that I can oversee fleet operations efficiently.
2. As a fleet manager, I want to view fleet-wide safety and fuel analytics so that I can improve overall performance.
3. As a fleet manager, I want to generate reports on driver behavior and fuel efficiency so that I can make informed operational decisions.

4.10 Real-time Notifications:

1. As a user, I want to receive real-time alerts and notifications so that I can respond immediately to important events.
2. As a user, I want my trip and profile data synchronized across devices so that my information is always up to date.

5 Functional Requirements

5.1 R1: User Management

5.1.1 R1.1 User Registration

R1.1.1: The system should allow users to create accounts using email and password authentication.
R1.1.2: The system should validate user credentials and enforce password security requirements.

5.1.2 R1.2 User Authentication

R1.2.1: The system should allow registered users to securely log into the application.
R1.2.2: The system should allow users to reset forgotten passwords securely.

5.1.3 R1.3 User Profile Management

R1.3.1: The system should allow users to view and update profile information.
R1.3.2: The system should allow users to manage saved contacts.

5.2 R2: Trip Tracking and Monitoring

5.2.1 R2.1: GPS Tracking

R2.1.1: The system should be able to track user trips and routes in real time using GPS services.
R2.1.2: The system should be able to display route information and trip progress on an interactive map.
R2.1.3: The system should be able to monitor trip speed, acceleration, braking, and cornering behavior.

5.2.2 R2.2: Sensor Data Collection

R2.2.1: The system should be able to collect accelerometer, gyroscope, and magnometer data during trips.
R2.2.2: The system should be able to monitor driving events such as harsh braking, rapid acceleration, and dangerous cornering.

5.2.3 R2.3 OBD Integration

R2.3.1: The system should be able to connect to Bluetooth-enabled OBD devices.
R2.3.2: The system should be able to retrieve vehicle diagnostic information, including RPM, coolant temperature, and diagnostic trouble codes.

5.3 R3: Driving Analysis and Safety

5.3.1 R3.1: Driving Behavior Analysis

R3.1.1: The system should be able to analyze driving patterns to identify unsafe driving behavior.
R3.1.2: The system should be able to generate per-trip and overall driver safety scores based on driving behavior.

5.3.2 R3.2: Risk Detection and Alerts

R3.2.1: The system should be able to detect possible crash events and dangerous driving conditions.
R3.2.2: The system should be able to provide real-time alerts for risky driving behavior.
R3.2.3: The system should be able to detect loud impact and screech events using available sensor and microphone data.
R3.2.4: The system should be able to detect fatigue indicators during long-distance trips.
R3.2.5: The system should be able to provide rest-stop alerts for extended driving sessions.

5.3.3 R3.3: Driver Personality Profiling

R3.3.1: The system should be able to classify driver behavior using AI-driven analysis.
R3.3.2: The system should be able to generate personalized driving improvement recommendations.

5.4 R4: Vehicle Health and Efficiency

5.4.1 R4.1: Vehicle Diagnostics

R4.1.1: The system should be able to monitor vehicle health using OBD diagnostic data.
R4.1.2: The system should be able to notify users about potential maintenance issues.

5.4.2 R4.2: Fuel Efficiency Monitoring

R4.2.1: The system should be able to calculate fuel efficiency statistics for trips.
R4.2.2: The system should be able to provide eco-driving recommendations to improve fuel consumption.
R4.2.3: The system should be able to estimate fuel usage per trip using driving behavior and vehicle diagnostic data.

5.5 R5: Gamification and Social Features

5.5.1 R5.1: Achievements and Challenges

R5.1.1: The system should be able to award badges and achievements based on driving performance.
R5.1.2: The system should be able to provide weekly driving challenges for users.

5.5.2 R5.2: Leaderboards

R5.2.1: The system should be able to display leaderboards based on safety, fuel efficiency, and driving consistency scores.
R5.2.2: The system should be able to compare user driving performance with users operating similar vehicle models.

5.5.3 R5.3 Emergency and Social Features

R5.3.1: The system should be able to notify trusted contacts during emergencies.
R5.3.2: The system should be able to allow family and friends to monitor active trips when authorized.

5.6 R6: Fleet Management

5.6.1 R6.1: Fleet Monitoring

R6.1.1: The system should allow fleet managers to monitor multiple drivers and vehicles.
R6.1.2: The system should be able to display fleet-wide trip and safety analytics.

5.6.2 R6.2: Fleet Reporting

R6.2.1: The system should be able to generate reports on driver performance and fuel efficiency trends.

5.7 R7: System Integration and Data Management

5.7.1 R7.1: Cloud Synchronization

R7.1.1: The system should synchronize user and trip data with cloud backend services.

5.7.2 R7.2: Real-time communication

R7.2.1: The system should provide real-time notifications using WebSocket communication.

5.7.3 R7.3: Maps Integration

R7.3.1: The system should be able to integrate with Azure Map services for route visualization and navigation.

5.7.4 R7.4: Data Storage

R7.4.1: The system should be able to securely store user, trip, and vehicle data in cloud databases.

6 API Service Contracts

Protocol: REST
Base URL:
Authorization: JWT

6.1 Auth Endpoints

6.1.1 POST /auth/register

```
POST /auth/register

Request Body: {
 "email": "String",
"username":"String",
"password": "String",
"name": "String",
"surname": "String",
"phone_number": "String",
"dob": "String",
"consent_status": "boolean"
}

Response 201: {
        "token": "String",
           "refresh_token": "String"
}

Response 409: {
        "error": "EMAIL_TAKEN | USERNAME_TAKEN | INVALID_PASSWORD|
            INVALID_PHONE | INVALID_DOB"
}

Response 422: {
        "error": "MISSING_FIELDS"
        "message": "Missing required fields"
}
```

6.1.2 POST /auth/login

```
Request Body: {
        "identifier": "email|username",
        "password": "string"
```

```
}

Response 200: {
        "token": "jwt_token_string",
        "refresh_token": "refresh_jwt"
        "expires_in": "int"
}

Response 401: {
        "error": "INVALID_CREDENTIALS",
           "message": "Incorrect Username/Email"
}
Response 404: {
        "error": "USER_NOT_FOUND",
        "message" : "User not found"
}
```

6.1.3 POST /auth/logout

```
Response 200: {
        "message": "Logged out successfully"
}

Response 401: {
        "error": "INVALID_CREDENTIALS",
           "message": "Incorrect Username/Email"
}

Response 404: {
        "error": "USER_NOT_FOUND",
        "message" : "user not found"
}
```

6.1.4 POST /auth/refresh

```
Request Body: {
        "refresh_token":"String"
}

Response 200: {
        "token": "access_jwt",
        "refresh_token": "refresh_jwt",
```

```
        "expires_in": "int"
}

Response 422: {
        "error": "MISSING_REFRESH_TOKEN"
        "Message": "Missing refresh token"
}

Response 401: {
        "error": "UNAUTHORIZED",
        "message": "Invalid refresh token"
}
```

6.2 Trips Endpoints

6.2.1 POST /trips/start_trip

```
Headers: {
"Authorization": "Bearer <jwt_token>"
}

Request Body: {
        "vehicle_id": "int",
        "start_time": "dateTime",
        "data_source": "OBD | PHONE",
        "start_location": {
                "lat": "long",
                "lng": "long"
}
}

Response 200 : {
        "message": "Trip started successfully",
        "data": {
                "trip_id": "uuid",
                "data_source": "OBD | PHONE"
}
}

Response 409 : {
        "error" : "TRIP_ALREADY_IN_PROGRESS",
        "message" : "Trip already in progress"
}
```

```
Response 403:{
        error : "UNAUTHORIZED"
        message: "user not found"
}

}
```

6.2.2 PATCH /trips/:trip_id/end_trip

```
Headers: {
"Authorization": "Bearer <jwt_token>"
}

Request Body: {
        "end_time": "dateTime",
        "route_polyline": "string",
        "distance_km": "float",
        "duration_minutes": "integer",
        "fuel_estimate_l": "float",
        "status": "COMPLETED | ABORTED",
        "scores":{
"safety_score": "float",
                        "eco_score": "float",
                        "overall_score":  "float"
}
}

Response 200: {
        "trip_id": "uuid",
        "username": "string",
        "status": "COMPLETED",
        "distance_km": "float",
        "duration_minutes": "integer",
        "fuel_estimate_l": "float",
        "scores": {
                "safety_score": "float",
                "eco_score": "float",
                "overall_score": "float"
        },
        "message": "Trip completed successfully"
}

Response 403: {
```

```
"error": "FORBIDDEN",
"message" : "You do not own this trip"
}

Response 404: {
"error": "TRIP_NOT_FOUND",
"message" : "Trip not found"
}

Response 409: {
"error": "TRIP_ALREADY_COMPLETED"
}
```

6.2.3 POST /trips/:trip_id/readings/record

```
Headers: {
"Authorization": "Bearer <jwt_token>"
}

Request Body:{
"recorded_at" : "dateTime",
"location" : {
        "longitutde" : "double",
        "latitude" : "double",
}
"data_source" : "OBD | PHONE",
"speed_kmh": 60.5,
"accelerometer": 0.12,
"gyroscope_x": 0.01,
"gyroscope_y": 0.00,
"gyroscope_z": -0.02,
"rpm": 2200,
"coolant_temp_c": 88.0,
"fuel_trim_percent": 1.5,
"throttle_position": 22.4,
"dtc_codes": ["P0420"]
}

Response 201

Response 404 : {
"error" : "NOT_FOUND",
```

```
}
```

6.2.4 GET /trips/:trip_id/summary

```
Headers: { "Authorization": "Bearer <jwt_token>"}

Response 200 : {
        "data" : {
        "trip_id" : "uuid",
        "vehicle_id" : "uuid",
"started_At" : "dateTime",
        "ended_At" : "dateTime",
        "status" : "string",
"data_source" : "OBD | PHONE | MIXED"
        "route_polyline" : "string" ,
"distance _km" : "float" ,
"duration_minutes" : "int",
"fuel_estimate" : "float" ,
"scores" : {
        "safety_score" : "float",
        "eco_score" : "float" ,
        "Overall_score" : "float"
},
"events" : [
{
"event_id" : "uuid",
        "event_type" : "string",
        "longitude" : "float",
        "latitude" : "float",
        "severity" : "float",
        "sensor_source" : "string",
        "time_stamp" : "dateTime"
}
                ]
}

}

Response 403 : {
        "error" : "FORBIDDEN"
"message" : "You do not own this trip"
}
```

```
Response 404 : {
        "error" :"NOT_FOUND"
"message" : "Trip not found"
}
Response 401 : {
        "error" : UNAUTHORIZED"
        "message" : "Can not view this trip"
}
```

6.2.5 GET /trips/history

```
Headers: {
        "Authorization": "Bearer <jwt_token>"
}


Query Parameters:
user_id = string
?start_date = date
?end_date = date        (defaults to now if not provided)
?status = COMPLETED | IN_PROGRESS | ABORTED

Response 200: {
"username": "string",
"start_date": "date",
"end_date": "date",
"total_trips": "integer",
"trips": [
  {
"trip_id": "uuid",
"start_date": "date",
"end_date": "date",
"distance_km": "float",
"duration_minutes": "integer",
"status": "string",
"data_source": "OBD | PHONE",
"scores": {
"safety_score": "float",
"eco_score": "float",
"overall_score": "float"
}
}
         ],
```

```
        "meta": {

        "count_trips": "int",
        "mean_distance": "float",
        "mean_duration": "float"
}
}

Response 400: {
        "error": "MISSING_FIELDS",
        "message": "Start date is required"
}

Response 400: {
        "error": "INVALID_DATE",
        "message": "Invalid end date"
}

Response 401: {
"error": "Unauthorized"
}
```

6.2.6 POST /trips/:trip_id/events/log

```
Headers: {
        "Authorization": "Bearer <jwt_token>"
}

Request Body: {
        "trip_id" : string;
        "event_type": "HARSH_BRAKE | HARSH_ACCELERATION |
           SHARP_CORNER
                        | CRASH_LIKE",
        "location" : {
"latitude": "float",
                "longitude": "float",
                }
        "severity": "float",
        "sensor_source": "ACCELEROMETER | GYROSCOPE | OBD",
        "timestamp": "dateTime"
}

Response 201: {
```

```
        "data":{
        "event_id": "uuid",
        "trip_id": "uuid",
        "type": "string",
        "severity": "float",
        "sensor_source": "string",
        "timestamp": "dateTime",
        "message": "Event logged successfully"
        }
}

Response 403: {
        "error": "‘FORBIDDEN",
        "message": "You do not own this trip"
}

Response 404: {
        "error": "TRIP_NOT_FOUND"
        "message": "Trip not found"
}

Response 400: {
        "error": "INVALID_EVENT_TYPE"
}

Response 401: {
"error": "UNAUTHORIZED"
}
```

6.2.7 GET /trips/live/:fleet_id

```
Headers : {
        "Authorization" : "Bearer <jwt_token>"
}

Response 200 : {
        "data" : {
                "trips" : [
                        "trip_id" : "uuid",
                        "username" : "String",
"current_location" : {
        "latitude" : "double",
```

```
        "longitude" : "double",
}
"last_updated": "dateTime",
"active_alert" : {
    "alert_id" : "uuid",
    "event_type" : "UNEXPECTED_STOP",
    "triggered_at" : "dateTime",
    "driver_response": "null | DRIVER_OK |      DRIVER_NEEDS_HELP"
  }
]
}
}
```

6.3 Contacts Endpoints

6.3.1 POST /contacts

```
Headers: {
"Authorization": "Bearer <jwt_token>"
}

Request Body: {
"identifier": "String"
}

Response 201: {
        "data": {
                "contact_id": "uuid",
                "username": "String"
}
}

Response 404: {
        "error": "USER_NOT_FOUND",
        "message":"User not found"
}

Response 409: {
        "error": "ALREADY_TRUSTED_CONTACT",
        "message": "Contact already added"
}

Response 400: {
```

```
        "error": "CANNOT_ADD_USER",
        "message": "Cannot add this user as a contact"
}

Response 401: {
        "error": "UNAUTHORIZED"
}
```

6.3.2 GET /contacts

```
Headers: {
"Authorization": "Bearer <jwt_token>"
}

Response 200: {
        "data": {
"contacts": [
        {
        "contact_id": "uuid",
        "username": "String",
        "name":"String",
        "email": "String"
},
]
},
"message": "Contacts successfully retrieved"
}

Response 401: {
        "error": "UNAUTHORIZED"
}

Response 403: {
        "error": "CANNOT_ACCESS_CONTACTS",
        "message":  "Cannot access these contacts"
}
```

6.3.3 POST /contacts/alerts

```
Headers: {
"Authorization": "Bearer <jwt_token>"
}
```

```
Request Body: {
        "event_type": "enum",
        "event_id": "uuid",
        "message": "String",
"contacts": [
                { "contact_id": "uuid" },
]
}

Response 400: {
        "error": "BAD_REQUEST",
"message": "Invalid request body"
}

Response 200: {
"message": "Contacts successfully alerted"
}

Response 401: {
        "error": "UNAUTHORIZED"
}

Response 403: {
        "error": "CANNOT_ACCESS_CONTACTS",
        "message":  "Cannot access these contacts"
}

Response 404: {
        "error": "CONTACT_NOT_FOUND",
        "message":  "Cannot find contact"
}

Response 404: {
        "error": "EVENT_NOT_FOUND",
        "message":  "Cannot find event"
}
```

6.3.4 POST /contacts/share_location

```
Headers: {
"Authorization": "Bearer <jwt_token>"
}
```

```
Request Body: {
        "contacts": {
                "trip_id": "uuid",
"contacts": [
                        { "contact_id": "uuid" },
]
}
}

Response 201: {
"message": "Location successfully shared",
"data": {
 "trip_id": "uuid",
"shared_with": [
{ "contact_id": "uuid",
"username": "String"
}
 ],
"shared_at": "Datetime"
}

Response 401: {
        "error": "UNAUTHORIZED"
}

Response 400: {
        "error": "NO_CONTACTS_PROVIDED",
        "message":  "Contact list is empty"
}

Response 403: {
        "error": "NOT_TRUSTED_CONTACT",
        "message":  "Cannot share location with non-trusted
           contacts"
}

Response 404: {
        "error": "TRIP_NOT_FOUND",
        "message":  "Cannot find trip"
}
```

6.4 Badges Endpoints

6.4.1 POST /badges/evaluate

```
Headers: {
"Authorization": "Bearer <jwt_token>"
}

Request Body : {
        "data":{
                "user_id" : "uuid",
                "trip_id" : "uuid"
}
}

Response 200: {
        "data" : {
"evaluated" : "boolean",
                "new_badges":
[
                        {
                                "badge_id" : "uuid",
                                "name" : "string",
                                "description" : "string",
                                "category" : "string",(enum)
                                "earned_at" : "dateTime",
                                "icon_url": "string"
                        }
                ]
},
"message" : "Badge evaluation complete"
}

Response 404 : {
        "error" : "NOT_FOUND"
        "message" : "Trip not found"
}
Response 401: {
        "error": "UNAUTHORIZED"
}
```

6.4.2 GET /badges

```
Headers: {
```

```
"Authorization": "Bearer <jwt_token>"
}


Response 200 : {
        "data" : {
                "earned" : [
                                {
                                        "badge_id" : "int",
                                        "name" : "string",
                                        "category" : "string",
                                        "description" : "string"
                                        "current" : "int",
                                }
                            ],
                "summary" : {
                        "Total_earned" : "int",
                        "categorys" : [],
}

}
}
```

6.4.3 GET /badges/definitions

```
Headers: {
"Authorization": "Bearer <jwt_token>"
}

Response 200 : {
        "data" : {
                "badges" : [
                        "badge_id" : "int",
                        "name" : "string",
                        "description" : "string",
                        "category" : "string",
                        "Icon_code": "string",
                        ]
                }
}
```

6.5 Leaderboard Endpoints

6.5.1 GET /leaderboard

```
Headers: {
"Authorization": "Bearer <jwt_token>"
}
Response 200 : {
        "Data" : {
  "category" : "string",
  "scope" : "string",
  "entries": [
    {
      "rank" : "int",
      "user_id": "uuid",
      "display_name": "String",
      "score" : "int"
    }
  ],
  "my_rank" : "int",
  "my_score" : "int"
}
}
```

7 Domain Model

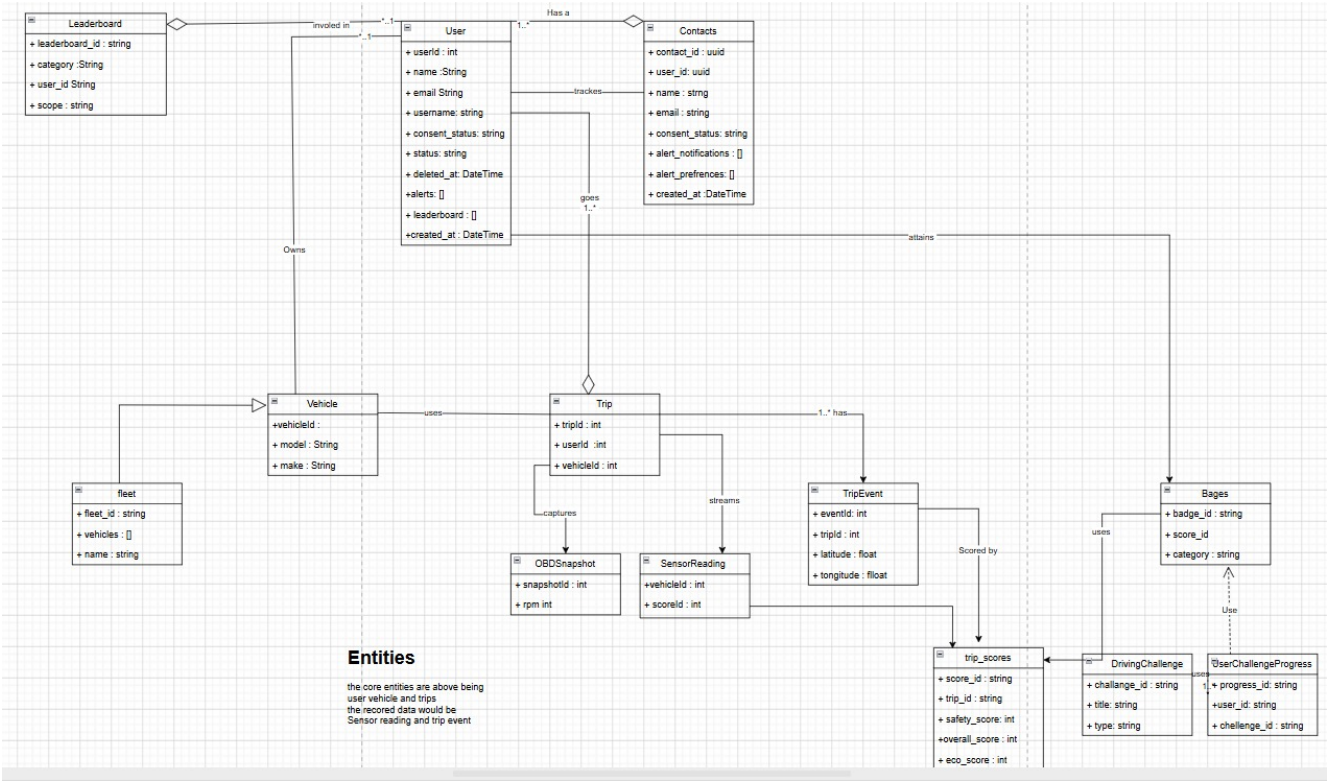

Figure 2: Domain Model with demo 1 use cases

8 Architectural Requirements

This section defines the architectural requirements for the Driving Tracker system. These requirements govern the structural decisions, quality attributes, design patterns, and constraints that guide development across the application.

8.1 Overall Software Architecture

This section discusses the high-level architecture of the Driving Tracker system across three levels of granularity, covering the primary architectural style, communication patterns between subsystems, and the architectural patterns applied within each subsystem.

8.2 Primary Architectural style

The Driving Tracker system adopts a Client-Server architecture as its primary architectural style. The system consists of two distinct client types: the Android mobile application, the web-based fleet management dashboard (and indirectly, OBD-II Bluetooth adapters as peripheral data sources). These clients interact with a centralised backend server responsible for data persistence, business logic, leaderboard computation, fleet coordination, and notification dispatch. This architecture is appropriate for Driving Tracker because the leaderboard, gamification, fleet management, and trusted contact features inherently require a shared server that multiple clients can communicate with.

8.3 Communication Patterns

Communication patterns used between subsystems:

8.3.1 Request-Response Model

The primary communication pattern between clients and the backend is the request-response model over HTTP. The Android application and the web dashboard send HTTP requests to the REST API, which processes the request and returns a JSON response. This pattern governs all standard operations including authentication, trip submission, leaderboard retrieval, vehicle management, and trusted contact alerts.

8.3.2 Push-Pull Model

A push-pull model is used for data synchronization and alerting. The Android application buffers trip and sensor data locally and pushes it to the backend in the background when connectivity is restored. Trusted contact alerts and safety notifications are pushed from the backend to the Android client. The web dashboard pulls fleet data from the backend on demand and on a configurable polling interval.

8.3.3 Event-Driven Messaging

A event-driven messaging pattern is used to process sensor and OBD data within the Android application. Raw sensor readings are emitted as events through Kotlin Flow streams. Downstream consumers/modules, including the scoring engine, harsh event detector, and UI layer, subscribe to these streams and react to readings independently. This decouples data production from data consumption and allows the sampling rate to differ from the processing and rendering rate.

8.4 Communication Protocols and Standards

The application-level communication protocols and standards applied across the system are as follows:

• REST over HTTPS is used for all client-to-server API communication. REST was selected over SOAP, GraphQL, and RPC approaches because it is stateless, widely supported across Android and web clients, and sufficient for the CRUD-heavy operations of this system. GraphQL was considered but ruled out given the system does not require dynamic field selection at the client level.
• HTTP/1.1 is the underlying transport protocol. TLS 1.2 or higher is required for all connections, enforced at the server and network layer.
• Bluetooth Classic is used for OBD-II communication between the Android device and the ELM327 adapter. This is more so a hardware constraint imposed by ELM327 adapters.
• JSON is the data interchange format for all REST API responses and requests.

8.5 Architectural Patterns Per Subsystem

The following architectural patterns are applied within each subsystem, with multiple combined where appropriate:

8.5.1 Backend API

• Layered (n-tier) Architecture: The backend is structured into three distinct layers: the routing layer, the controller layer, and the service layer. Each layer has a single responsibility and communicates only with the layer directly below it.
• Service-Oriented Architecture (SOA): Backend functionality is decomposed into independently operable services including authentication, trip management, gamification, fleet management, notifications, and user management. Each service exposes a well-defined interface.

8.5.2 Android Application

• Modular Architecture: The Android application is divided into independently developed and tested feature modules covering sensors, OBD communication, trip tracking, gamification, notifications, and data synchronization.
• Offline-First Architecture: The application treats the local Room database as the primary data store. All writes are committed locally before being synchronized to the backend, ensuring uninterrupted trip recording in low-connectivity or completely offline environments.
• Event-Driven Architecture: The sensor pipeline uses reactive event streams implemented with Kotlin Flow, allowing multiple subsystems to consume sensor data independently without tight coupling.

8.5.3 Web Dashboard

• Component-Based Architecture: follows The React and Next.js web dashboard a component-based UI architecture, decomposing screens into reusable, independently testable UI components with centralized state management.

8.6 High-Level Architecture Diagram

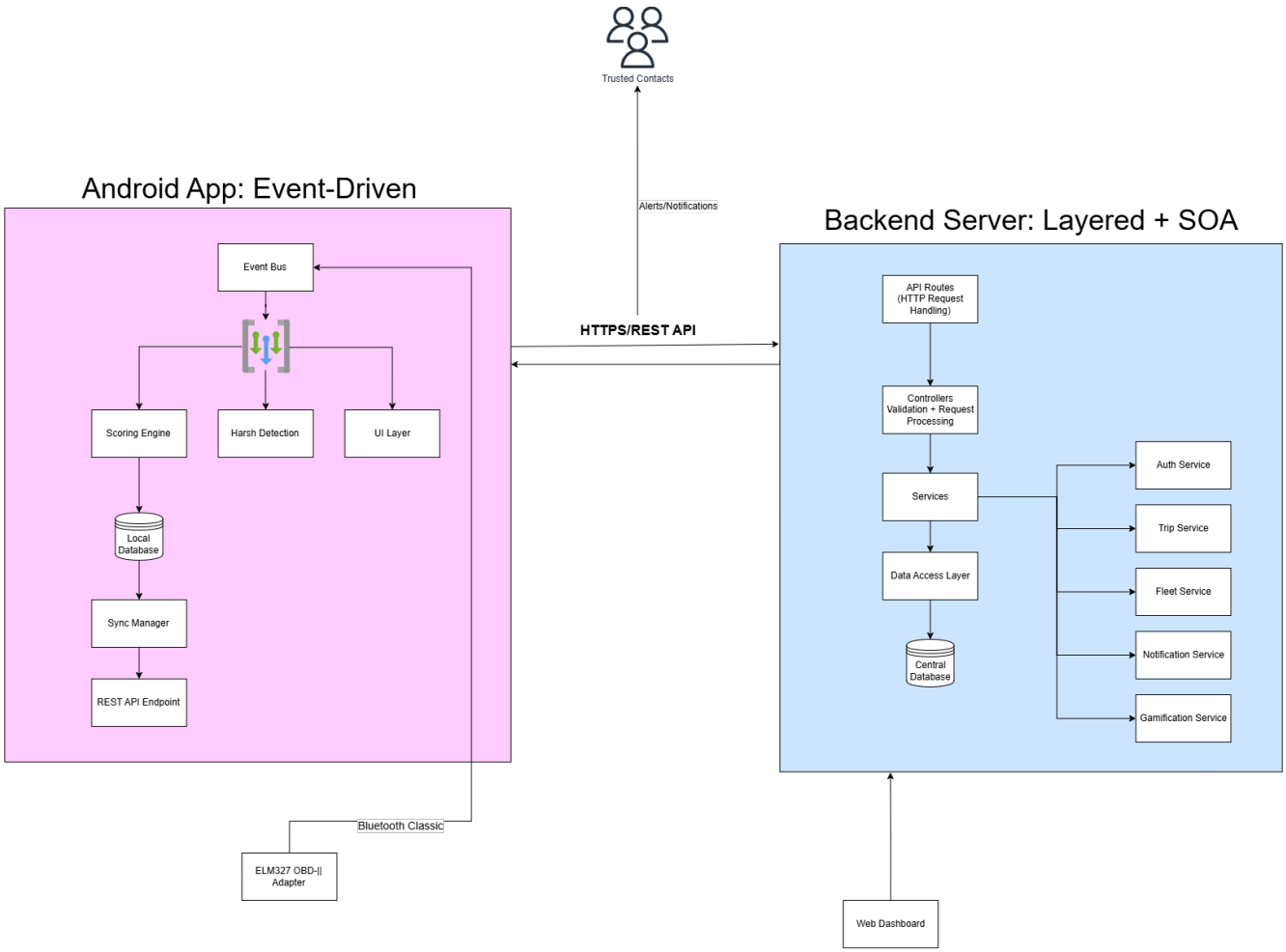

Figure 3: Architectural Diagram

8.7 Overall Architectural Quality Requirements

8.7.1 Performance

• Real-time sensor and OBD-II data must be processed and reflected in the UI within 500 milliseconds to avoid degrading the driving experience or navigation responsiveness.
• Trip scoring computations must be completed within 2 seconds of a trip ending.
• The backend API must support a minimum of 500 concurrent users, to account for standard users and fleets, with average response times under 300 milliseconds for standard read operations.

8.7.2 Reliability

• The system must achieve a minimum uptime of 99.5• Trip data must not be lost if the device loses connectivity during the trip. All sensor and OBD data must be stored locally

and synced to the backend once connectivity is restored, and when a trip has ended.
• The application must handle Bluetooth OBD adapter disconnections gracefully, continuing trip recording using phone sensors alone and notifying the user of the disconnection.
• Background services must recover automatically from crashes without requiring user intervention considering the app will be used by people driving.

8.7.3 Scalability

• The backend must scale horizontally to accommodate fleet management workloads where hundreds of vehicles may be transmitting data simultaneously.
• The leaderboard and gamification engine must handle ranking computations across a growing user base without degrading query performance, using pagination and caching where appropriate.
• The database schema must support both individual users and fleet operators under the same backend infrastructure.

8.7.4 Security

• All API communication must be conducted over HTTPS using TLS 1.2 or higher.
• User authentication must use short-lived JWT access tokens with refresh token rotation. Refresh tokens must be stored server-side and invalidated on logout.
• Passwords must be hashed using bcrypt with a minimum cost factor of 10 before storage. • Encryption of tokens on the android app level before storage.
• Role-based access control must be enforced at the API level, distinguishing between standard users, family group members, fleet managers, and administrators.
• Location data and trip history are classified as sensitive personal data and must be stored with restricted access, in compliance data protection regulations.
• Users must provide explicit consent before location tracking, including in the background, is activated. Consent status must be recorded and enforced.
o Security can be measured by the standards for encryption and hashing being used for stored tokens and passwords respectively. OWASP Mobile Application Security project can serve as a guideline and quantifier for the android app's security.

8.7.5 Maintainability

• A modular architecture should be employed, with clearly separated concerns across sensor processing, OBD communication, trip tracking, gamification, notifications, and authentication.
• All backend modules must expose well-defined interfaces to allow independent replace-

ment or extension without affecting other modules.
• The system must include a CI/CD pipeline that runs automated tests and linting on every push, and deploys to staging on merge to the main branch.
• Code must follow consistent naming conventions(snake case) and be documented at the module and function level.

8.7.6 Usability and Accessibility

• The Android application must function correctly on devices running Android 9 (API level 28) and above.
• The UI must remain responsive and unblocked during all background operations including sensor sampling, OBD polling, and network sync.
• Driving alerts and notifications must be non-intrusive and must not require the user to interact with the phone while driving.

8.7.7 Portability

• The backend must be deployable to Azure App Service or Azure Functions without code changes.
• The web dashboard must be compatible with modern browsers including Chrome, Edge, Firefox, and Safari.
• OBD integration must support any ELM327-compatible Bluetooth adapter, not a specific proprietary device.

8.8 Design Patterns

8.8.1 Observer Pattern

Sensor data streams on Android are modelled as observable flows using Kotlin Flow. UI components and scoring logic subscribe to these flows and react to new readings. This pattern decouples the sensor sampling ratee from the UI refresh rate and allows multiple consumers to receive the same sensor events independently.

8.8.2 Strategy Pattern

Driver scoring and harsh event detection could use configurable strategies that encapsulate the scoring algorithm. Different vehicle types or driving profiles may use different scoring strategies without altering the core trip recording logic. This also provides an easier path to introduce ML based scoring while alongside rule-based scoring.

8.9 Constraints

8.9.1 Hardware Constraints

• OBD-II diagnostics require a physical ELM327-compatible Bluetooth adapter connected to the vehicle. Vehicles made before 1996 may not have an OBD-II port and will not support this feature.
• Sensor availability varies between Android devices. The system must implement adaptive thresholds and degrade gracefully on devices that do not have specific sensors.
• Bluetooth Classic is required for ELM327 communication. Although most devices do have Bluetooth support, those without cannot use OBD features.

8.9.2 Platform Constraints

• Background location access requires permission which needs to be explicitly granted by the user on Android 10 and above.
• Background execution is subject to Android battery optimization restrictions. Ideally, the system should use a foreground service with a persistent notification during active trip recording to prevent the system from terminating the process.

8.9.3 Real-Time Processing Constraints

• Road noise, vibration, and environmental interference could produce false positives in harsh event detection. Configurable sensitivity levels, as well as other techniques like smoothing, should be utilised to reduce false alert rates.
• Sensor and OBD data processing must not block the main thread, to avoid delaying UI or navigation. Processing must occur on background threads or coroutines.

8.9.4 Privacy and Legal Constraints

• Location tracking must not begin until the user has granted location permissions and provided consent through the in-app consent flow.
• Live trip sharing with trusted contacts must be managed per trip. Contacts must not be able to view location data without the user actively sharing their location.

8.9.5 Technology Constraints

• Backend infrastructure must be deployable to Microsoft Azure.
• Background execution is subject to Android battery optimization restrictions. Ideally,

the system should use a foreground service with a persistent notification during active trip recording to prevent the system from terminating the process.

8.10 Architectural Responsibilities

The architectural responsibilities of the Driving Tracker system are the cross-cutting concerns that must be addressed at the system level before being delegated to individual architectural components. The following responsibilities have been identified:

Responsibility	**Description**
AuthenticationManager	Manages user identity verification through JWT-based access and refresh tokens. Responsible for token issuance, rotation, validation, and invalidation across all clients.
AuthInterceptor	Handles attachment of tokens to requests and token refreshing on token expiry seamlessly.
SessionManager	Manages user sessions including refresh token storage, expiry tracking, and session invalidation on logout or suspicious activity.
SyncManager	Coordinates offline data buffering on the Android client and background synchronisation of trip records, sensor readings, and OBD data to the backend when connectivity is restored.
NotificationManager	Responsible for dispatching safety alerts, trusted contact notifications, and driving reminders across the Android push notification system.
EventBus	Manages the internal event-driven pipeline on the Android application, routing sensor and OBD events to the appropriate consumers including the scoring engine, harsh event detector, and UI layer.
ApiManager	Handles all requests to the API, providing one centralised communication point.
RoomManager	Manages local storage of leaderboard data, trip summaries, and user profiles to reduce redundant API calls and allow for offline use.

Table 1: Architectural Responsibilities

8.11 Backend REST API

The Backend REST API is the central component of the system. It serves as the single data source for all clients, enforces business rules and logic, and manages authentication and authorisation.

8.11.1 Requirements

Performance The backend must respond to API requests quickly enough to support real-time data submission and live fleet monitoring.

- Standard endpoints should respond within 300–500 milliseconds under normal circumstances.
- Trip data submission endpoints must acknowledge receipt immediately.

Security The backend is the enforcement point for all authentication and authorisation decisions.

- JWT-based authentication with access and refresh token rotation.
- Role-based access control enforced at the route middleware level.
- Parameterised queries through the ORM to prevent SQL injection.

Maintainability The backend must be structured to allow individual service modules to be modified independently without affecting unrelated parts of the system.

- Three-layer architecture separating routing, controllers, and services.
- Each domain (authentication, trips, gamification, fleet, users) has its own service module.
- Consistent error handling and response formatting across all endpoints.

Testability The service layer must be independently testable without requiring an active HTTP server or database connection.

- Unit test coverage above 70% for all service functions.
- Integration tests covering all API endpoints.
- A test database seeded with simulated trip and OBD data for integration test scenarios.

8.11.2 Frameworks and Technologies

Criterion	Node.js + Express	Deno + Oak	Bun + Hono
Ecosystem maturity	Very mature, large ecosystem	Maturing, smaller ecosystem	Emerging, limited ecosystem
TypeScript support	Via ts-node / tsc	Native	Native
Azure support	First-class	Limited	Limited
Community and libraries	Largest	Moderate	Small
Production readiness	Very high	Moderate	Low

Table 2: Runtime and Web Framework Comparison

Runtime and Web Framework

Criterion	Prisma	TypeORM	Drizzle
Type safety	Excellent, auto-generated types	Good	Excellent
Schema migrations	Automated	Manual or automated	Manual
PostgreSQL support	Full	Full	Full
Learning curve	Low	Moderate	Moderate
Community	Large and active	Large	Growing

Table 3: ORM Comparison

ORM

Criterion	Zod	Joi	class-validator
TypeScript integration	Native, infers types	Separate types needed	Decorator-based
Runtime validation	Yes	Yes	Yes
Bundle size	Small	Moderate	Moderate
Error messages	Detailed and customisable	Good	Good
Composability	Excellent	Good	Moderate

Table 4: Schema Validation Comparison

Schema Validation

8.11.3 Technology Choice

- **Node.js with Express and TypeScript** is selected as the backend runtime and web framework. Express is a minimal, well-understood framework with first-class Azure support, a vast library ecosystem, and extensive community resources. Its stateless request handling model aligns directly with the horizontal scaling requirement. TypeScript provides static type safety across the codebase, reducing runtime errors and improving maintainability.
- **Prisma** is selected as the ORM due to its automatic type generation from the database schema, automated migration management, and excellent PostgreSQL support. The auto-generated types eliminate an entire class of type mismatch bugs at the database boundary.
- **Zod** is selected for schema validation due to its native TypeScript integration. Zod schemas serve as both the runtime validation logic and the compile-time type source, eliminating the need to maintain separate type definitions for request bodies.
- **PostgreSQL** is selected as the database for its robustness, support for complex relational queries required by leaderboard computations, and native support for geospatial data types relevant to route storage.

8.12 Android Application

The Android application is the primary client of the system. It is responsible for collecting sensor and OBD data during trips, processing raw readings into driving scores and events, and presenting them to the user.

8.12.1 Reliability

- Trip data must not be lost during network outages or application crashes. Room provides the durability guarantee.
- Bluetooth OBD disconnections must be handled gracefully with automatic fallback to phone-sensor-only recording.
- Background services must restart automatically if terminated by the Android system.

8.12.2 Security

- JWT tokens and refresh tokens stored on the device must be encrypted using the Android Keystore system before being written to persistent storage.
- Location and sensor data stored locally must not be accessible to other applications.

8.12.3 Usability

- The UI must always be responsive. No user-facing operation may block the main thread.
- Driving alerts must be delivered as non-intrusive notifications that do not require screen interaction while driving.
- The application must support devices running Android 9 and above and must adapt layouts to varying screen sizes.

8.12.4 Integrability

- The application must integrate with Google Maps for route visualisation.
- The application must integrate with ELM327-compatible OBD adapters over Bluetooth Classic.
- The application must communicate with the backend REST API using standard HTTP with JSON payloads.

8.12.5 Frameworks and Technologies

Criterion	**Jetpack Compose**	**XML Views + ViewBinding**	**Flutter**
Language	Kotlin native	Kotlin/Java	Dart (separate language)
Reactivity model	State-driven recomposition	Manual view updates	Widget tree rebuilds
Google support	Primary recommended	Legacy, maintained	Separate product
Sensor integration	Native Android APIs	Native Android APIs	Platform channels needed
Maturity	Stable	Very mature	Stable

Table 5: UI Framework Comparison

UI Framework

Criterion	Room	SQLite (direct)	Realm
Type safety	Compile-time verified DAOs	None	Good
Coroutine support	Native Flow support	Manual	Native
Migration support	Built-in	Manual	Built-in
Google support	Jetpack component	None	Third party
Community	Large	Large	Moderate

Table 6: Local Database Comparison

Local Database

Criterion	Retrofit + OkHttp	Ktor Client	Volley
Type safety	Interface-based, type-safe	Type-safe DSL	None
Coroutine support	Native suspend functions	Native	Callback-based
Interceptor support	OkHttp interceptors	Plugin-based	Limited
Community	Very large	Growing	Declining
Production readiness	Very high	High	Moderate

Table 7: Networking Framework Comparison

Networking

8.12.6 Technology Choice

- **Kotlin with Jetpack Compose** is selected for the Android application. Kotlin is the officially recommended language for Android development and provides native coroutine support essential for the concurrent sensor, OBD, and networking operations this system requires. Jetpack Compose provides a modern, reactive UI framework that eliminates the manual view synchronisation required by the XML view system, reducing UI-related bugs and accelerating development.
- **Room** is selected as the local database because it provides compile-time verification of SQL queries, native Kotlin Flow support for reactive data observation, and built-in migration management. These properties directly support the offline-first architecture requirement.
- **Retrofit with OkHttp** is selected for networking because its interface-based design produces type-safe API clients that map directly to the backend REST API,

and OkHttp interceptors provide a clean mechanism for transparent JWT token attachment and refresh.

9 Technology Requirements

9.1 TR1 - Mobile Application Runtime:

Android device running Android 8.0 or higher

9.2 TR2 - Backend Runtime:

Node.js version 20 or higher

9.3 TR3 - Database:

PostgreSQL version 15 or higher
Prisma ORM
Redis (Realtime Updates)

9.4 TR4 - Containerization:

Docker Desktop with Docker Compose v2 or higher

9.5 TR5 - Cloud Infrastructure:

Azure app service
Azure DB for PostgreSQL Flexible Server

9.6 TR6 - Frontend Runtime:

Next.js 15

9.7 TR7 - Package Management:

Yarn 4 via Corepack

9.8 TR8 - Version Control:

Git version 2.x or higher

9.9 TR9 - Android Development Environment:

Android Studio
Retrofit (HTTP client for Android and Java)

9.10 TR10 - External APIs and Services:

Azure Maps Web SDK
Bluetooth OBD-||

9.11 TR11 - Authentication:

JSON Web Tokens

9.12 TR12 - CI/CD Pipeline:

Github actions